

Wettbewerbsrecherche mit cintel

Handbuch zum Competitive Intel Research Tool

Was es tut	Findet Unternehmen, liest ihre Webseiten und traegt Unternehmens- und Produktdaten in die Competitive Intel Master DB ein
Fuer wen	Alle im Team. Programmierkenntnisse sind nicht noetig, Erfahrung mit der Tastatureingabe von Befehlen auch nicht - das wird in Kapitel 2 von Grund auf erklart
Repository	github.com/OCP-69/260818_Business-Intell_Comp-research_Tool
Projektordner	C:\Users\olafp\Desktop\Arbeitsordner\260818_Business Intell_Comp research_Tool
Stand	18. August 2026

Dieses Handbuch setzt keinerlei Vorwissen voraus. Jeder Befehl ist vollstaendig abgedruckt und kann eins zu eins kopiert werden.

Inhalt

1. **Was dieses Tool ueberhaupt macht** — Das Grundprinzip in fuenf Minuten
2. **Das Terminal - wo Sie die Befehle eintippen** — Von Grund auf, mit erster Uebung
3. **Wo alles liegt und was sich womit abgleicht** — Drei Orte, zwei Abgleichverfahren
4. **Wo Sie mit der Recherche anfangen** — Der richtige erste Schritt
5. **Voraussetzungen** — Was einmalig eingerichtet sein muss
6. **Das Tool starten** — Schritt fuer Schritt zum ersten Lauf
7. **Die Ergebnisse verstehen** — Welche Datei was enthaelt
8. **Qualitaet und Verhalten anpassen** — Wenn Ihnen etwas nicht gefaellt
9. **Spalten ergaenzen** — Die Master-Tabelle erweitern
10. **Laeufe automatisieren** — Profile und Zeitsteuerung
11. **Aenderungen ueber GitHub einbringen** — Branch, Commit, Pull Request
12. **Fehlermeldungen nachschlagen** — Was die Meldungen bedeuten
13. **Befehlsuebersicht** — Alles auf einen Blick

Wenn Sie noch nie einen Befehl eingetippt haben

Lesen Sie Kapitel 2 vollstaendig und machen Sie die kleine Uebung am Ende. Danach sind alle uebrigen Kapitel bedienbar. Ueberspringen Sie Kapitel 2 nicht - dort steht, in welches Fenster die Befehle gehoeren.

1. Was dieses Tool ueberhaupt macht

Es gibt eine zentrale Excel-Datei, die **Competitive Intel Master DB**. Darin steht, welche Unternehmen im Markt fuer uns relevant sind, was sie anbieten und wie wir sie einschaeetzen. Diese Datei von Hand zu pflegen ist muehsam: man muesste jede Firmenwebseite einzeln aufrufen, die Produkte heraussuchen und alles abtippen.

Genau das erledigt **cintel**. Sie sagen dem Tool, welche Firmen oder welches Marktsegment Sie interessieren. Das Tool ruft die Webseiten auf, liest sie, sortiert die Informationen und schreibt sie in eine neue Fassung der Excel-Datei.

So sieht das Ergebnis in der Tabelle aus

Jede Firma bekommt **eine Zeile fuer das Unternehmen selbst** und darunter **je eine Zeile pro Produkt**. Alle Zeilen einer Firma teilen sich dieselbe Company_ID:

Company_ID	Company & Product	Company	Product_name	Was in der Zeile steht
42	Company information	Autodesk	(leer)	Gruendungsjahr, Mitarbeiterzahl, Standort, Umsatz
42	Product	Autodesk	Revit	Kategorie, USP, Preismodell, Nachhaltigkeit
42	Product	Autodesk	Fusion 360	dasselbe, aber fuer dieses Produkt

Die fuenf Arbeitsschritte des Tools

Das Tool arbeitet immer in derselben Reihenfolge. Sie muessen das nicht steuern - es hilft nur beim Verstehen der Bildschirmausgabe:

Schritt	Was passiert
1. Finden	Welche Firmen bearbeitet werden - entweder bekannte Firmen mit Luecken oder neue Firmen aus einer Websuche
2. Abgleichen	Ist die Firma schon in der Tabelle? Verglichen wird ueber die Internetadresse und den Firmennamen
3. Webseite lesen	Startseite aufrufen, dann Produkt- und Loesungsseiten. Hier wird auch geprueft, ob die Firma echt ist
4. Auswerten	Aus dem Seitentext werden die Tabellenfelder gefuellt
5. Eintragen	Die Zeilen kommen in eine neue Fassung der Excel-Datei

Die wichtigste Sicherung

In Schritt 3 prueft das Tool, ob die Webseite wirklich erreichbar ist und ob der Firmenname darauf vorkommt. Beim Aufbau lieferte die Websuche eine Firma mit falscher Internetadresse - ohne diese Pruefung waere eine Karteileiche in der Tabelle gelandet. Firmen, die durchfallen, werden mit Begruendung in einer eigenen Datei aufgelistet.

Ihre Originaldatei wird nie veraendert

Das Tool liest die Master-Datei nur. Geschrieben wird immer eine **neue Datei mit hoeherer Versionsnummer** in einem Ausgabeordner auf Ihrem Rechner. Sie koennen also nichts kaputtmachen.

2. Das Terminal - wo Sie die Befehle eintippen

Alle Befehle in diesem Handbuch werden in ein bestimmtes Fenster getippt. Dieses Fenster heisst **Terminal**. Wenn Sie damit noch nie gearbeitet haben: kein Problem, dieses Kapitel erklart es von Grund auf.

2.1 Was ein Terminal ist

Normalerweise bedienen Sie den Computer mit der Maus: klicken, ziehen, Menues aufklappen. Ein Terminal ist der andere Weg - Sie **schreiben dem Computer auf, was er tun soll**, und druecken die Eingabetaste.

Stellen Sie sich den Unterschied so vor: Mit der Maus zeigen Sie auf Dinge im Regal. Im Terminal schreiben Sie einen Bestellzettel. Der Zettel ist unhandlicher, aber Sie koennen darauf Dinge bestellen, fuer die es gar keinen Knopf gibt - und Sie koennen denselben Zettel jederzeit wieder verwenden. Genau das brauchen wir hier.

Es ist ein ganz normales Fenster

Das Terminal laesst sich verschieben, vergroessern und mit dem X oben rechts schliessen wie jedes andere Fenster. Es sieht nur ungewohnt aus, weil es fast nur aus Text besteht.

2.2 Welches Terminal - und wie Sie es oeffnen

Windows bringt mehrere solcher Fenster mit. Wir nutzen durchgaengig **Windows PowerShell**. Bitte kein anderes - die Skripte in Kapitel 10 laufen nur dort.

So oeffnen Sie es:

- 1 Druecken Sie die **Windows-Taste** auf der Tastatur (die mit dem Fenster-Symbol, unten links neben der Leertaste). Das Startmenue geht auf, der Schreibcursor steht im Suchfeld.
- 2 Tippen Sie **powershell**. Sie muessen nirgends hinklicken - die Suche laeuft mit.
- 3 In der Trefferliste erscheint oben **Windows PowerShell**. Druecken Sie die **Eingabetaste** oder klicken Sie den Treffer an.

Es oeffnet sich ein Fenster mit dunklem oder hellem Hintergrund. Darin steht eine Zeile wie diese, mit einem blinkenden Strich am Ende:

```
PS C:\Users\olafp>
```

Diese Zeile heisst **Eingabeaufforderung**. Sie bedeutet: *Ich bin bereit, sag mir was*. Der blinkende Strich zeigt, wo Ihre Eingabe erscheint. Alles, was Sie jetzt tippen, landet hinter dem Groesser-als-Zeichen.

2.3 Der wichtigste Begriff: das aktuelle Verzeichnis

In dem Text `PS C:\Users\olafp>` steckt eine Information, die spaeter ueber Erfolg und Misserfolg entscheidet: **C:\Users\olafp** ist der Ordner, in dem das Terminal gerade *steht*.

Das ist so, als stuenden Sie in einem Aktenschrank vor einer bestimmten Schublade. Sagen Sie *"gib mir die Mappe Mueller"*, wird in genau dieser Schublade gesucht - nicht im ganzen Schrank. Steht das Terminal im falschen Ordner, findet es unsere Befehle nicht, obwohl alles korrekt installiert ist.

Die haeufigste Fehlerursache ueberhaupt

Fast alle Meldungen der Art *"kann nicht gefunden werden"* kommen daher, dass das Terminal im falschen Ordner steht. Pruefen Sie das immer zuerst.

2.4 In den Projektordner wechseln

Der Befehl zum Wechseln heisst `cd` - kurz fuer *change directory*, Verzeichnis wechseln. Dahinter kommt der Zielordner in Anfuhrungszeichen:

```
cd "C:\Users\olafp\Desktop\Arbeitsordner\260818_Business Intell_Comp research_Tool"
```

- **Warum die Anfuhrungszeichen?** Der Pfad enthaelt Leerzeichen ("Business Intell"). Ohne Anfuhrungszeichen haelt das Terminal jedes Leerzeichen fuer das Ende des Pfades und meldet einen Fehler. Kopieren Sie die Zeile deshalb immer mitsamt beiden Anfuhrungszeichen.
- **Gross- und Kleinschreibung** ist bei Ordernamen egal.
- Hat es geklappt, aendert sich die Eingabeaufforderung - sie zeigt jetzt den neuen Ordner an. Es erscheint **keine** Erfolgsmeldung. Beim Terminal gilt: keine Nachricht ist eine gute Nachricht.

Danach sieht die Zeile so aus:

```
PS C:\Users\olafp\Desktop\Arbeitsordner\260818_Business Intell_Comp research_Tool>
```

2.5 Text ins Terminal kopieren

Sie muessen nichts abtippen. Aus diesem PDF heraus geht es so:

- 1 Den Befehl im PDF mit der Maus markieren und **Strg + C** druecken.
- 2 In das PowerShell-Fenster klicken, damit es aktiv ist.
- 3 **Strg + V** druecken. Alternativ genuegt ein **Rechtsklick** - in PowerShell fuegt der rechte Mausklick direkt ein.
- 4 **Eingabetaste** druecken. Erst jetzt wird der Befehl ausgefuehrt.

Ein Befehl laeuft erst mit der Eingabetaste

Solange Sie nicht Enter gedrueckt haben, koennen Sie den Text noch korrigieren oder mit der Ruecktaste ganz loeschen. Nichts ist passiert.

2.6 Woran Sie erkennen, dass ein Befehl fertig ist

Waehrend ein Befehl laeuft, erscheinen Textzeilen, und Sie koennen nichts eintippen. **Fertig ist er, wenn die Eingabeaufforderung wieder erscheint** - also wieder eine Zeile mit `PS . . .>` und blinkendem Strich da ist.

Ein Recherchelauf braucht mehrere Minuten. Das Fenster sieht dabei zeitweise aus, als haenge es. Das ist normal. Lassen Sie es einfach stehen und warten Sie.

Situation	Was zu tun ist
Der Befehl laeuft und laeuft	Warten. Ein Lauf ueber 5 Firmen dauert 3 bis 5 Minuten

Situation	Was zu tun ist
Sie wollen abbrechen	Strg + C druecken. Der Lauf stoppt sofort. Bereits geschriebene Dateien bleiben erhalten
Sie sind fertig fuer heute	Fenster mit dem X schliessen oder <code>exit</code> eintippen
Sie haben sich vertippt	Ruecktaste zum Loeschen, oder Esc loescht die ganze Zeile

2.7 Erste Uebung

Bevor Sie richtig loslegen, machen Sie diese drei Schritte einmal durch. Sie dauern zwei Minuten und geben Sicherheit.

Uebung 1 — Wo stehe ich gerade?

```
pwd
```

Antwort ist der Ordner, in dem das Terminal steht. `pwd` steht fuer *print working directory*.

Uebung 2 — In den Projektordner wechseln

```
cd "C:\Users\olafp\Desktop\Arbeitsordner\260818_Business Intell_Comp research_Tool"
```

Uebung 3 — Was liegt hier?

```
ls
```

Es erscheint eine Liste. Darin muessen unter anderem `cintel`, `config`, `docs` und `README.md` auftauchen. Sehen Sie diese Namen, sind Sie am richtigen Ort und alles Weitere funktioniert.

Sie muessen den `cd`-Befehl bei jedem neuen Fenster wiederholen

Das Terminal merkt sich den Ordner nur, solange das Fenster offen ist. Oeffnen Sie es am naechsten Tag neu, steht es wieder in `C:\Users\olafp`. Der erste Befehl in einem frischen Fenster ist deshalb immer der `cd`-Befehl.

3. Wo alles liegt und was sich womit abgleicht

Es gibt **drei Orte**, an denen etwas zu diesem Projekt liegt. Wer sie durcheinanderbringt, sucht Dateien an der falschen Stelle. Zwei davon gleichen sich automatisch ab, einer nur auf Befehl.

3.1 Die drei Orte

Ort	Was dort liegt	Abgleich
1. Laufwerk H: H:\Geteilte Ablagen\Lo opForgeLab\...	Die offizielle Master-Tabelle des Teams. Der gemeinsame Stand, den alle sehen	Automatisch. H: ist Google Drive. Aenderungen erscheinen ohne Ihr Zutun bei allen Kollegen
2. Projektordner C:\Users\olafp\Desktop \Arbeitsordner\...	Das Programm selbst und Ihre Laufergebnisse im Unterordner data	Nur auf Befehl. Nichts verlaesst diesen Ordner, solange Sie es nicht anstossen
3. GitHub github.com/OCP-69/2608 18_...	Ausschliesslich der Programmcode und dieses Handbuch. Keine Wettbewerbsdaten	Nur auf Befehl. Ueber die Git-Befehle aus Kapitel 11

3.2 Zwei verschiedene Abgleichverfahren

Das ist der Punkt, an dem die meisten Missverstaendnisse entstehen. Die beiden Verfahren haben nichts miteinander zu tun:

	Google Drive (Laufwerk H:)	Git und GitHub
Was wird abgeglichen	Dateien - Excel, Dokumente, Bilder	Programmcode und Handbuch
Wer stoest es an	Niemand. Es laeuft im Hintergrund	Sie selbst, mit einem Befehl
Wie schnell	Sekunden bis wenige Minuten	Sofort, aber eben erst auf Befehl
Woran erkennbar	Ein Google-Drive-Symbol unten rechts in der Taskleiste	<code>git status</code> zeigt es an

Der Projektordner wird NICHT automatisch mit GitHub abgeglichen

Wenn Sie eine Datei im Projektordner aendern, passiert auf GitHub zunaechst gar nichts. Erst die Befehle aus Kapitel 11 laden sie hoch. Umgekehrt genauso: aendert ein Kollege etwas auf GitHub, merken Sie das erst, wenn Sie `git pull` ausfuehren.

3.3 Wo arbeiten Sie eigentlich?

Immer im Projektordner auf C:. Dort steht das Terminal, dort laufen die Befehle, dort entstehen die Ergebnisse. Auf H: greifen Sie nur lesend zu, und GitHub sehen Sie nur im Browser oder ueber Git-Befehle.

Der Weg einer Datei durch die drei Orte sieht so aus:

```

H:\ Master-Tabelle  ---lesen--->  Projektordner C:\
                                   |
                                   v
                        data\outputs\...\v2.3.xlsx  (Ergebnis)
                                   |
                        Sie pruefen es und kopieren es
                                   |
                                   v
                        H:\ ... \Competitors_DB\  (Team sieht es)

```

3.4 Ein typischer Arbeitsablauf

- 1 Terminal oeffnen und in den Projektordner wechseln (Kapitel 2).
- 2 Recherchelauf starten. Grundlage ist die Master-Tabelle - entweder direkt von H: oder die zuletzt bereinigte Fassung aus data\outputs.
- 3 Ergebnis pruefen: die neue Excel-Datei ansehen und den Bericht report.md lesen (Kapitel 7).
- 4 **Erst wenn Sie zufrieden sind:** die neue Datei nach H: kopieren, damit das Team sie sieht. Das ist ein bewusster Schritt und passiert nicht von selbst.
- 5 Haben Sie am Programm etwas geaendert, laden Sie das ueber Git nach GitHub hoch (Kapitel 11). Ergebnisdateien gehoeren **nicht** dorthin.

Ergebnis ins Team-Laufwerk uebernehmen

Am einfachsten mit dem Explorer: beide Ordner nebeneinander oeffnen und die Datei hinueberziehen. Wer lieber tippt, merkt sich das Ziel zuerst unter einem kurzen Namen und kopiert dann - beide Zeilen nacheinander, jeweils mit Eingabetaste:

```
$ziel = "H:\Geteilte Ablagen\LoopForgeLab\A_Strategy\02_Deliverables\Business_Competitor_Intel\Competitors_DB"
```

```
copy "data\outputs\Competitive_Intel_Master_DB_v2.3.xlsx" $ziel
```

Die erste Zeile legt nur den Zielpfad ab und gibt nichts aus. Erst die zweite kopiert. Der Name \$ziel gilt, solange das Fenster offen bleibt.

Warum die Ergebnisse nicht gleich auf H: geschrieben werden

Ein Lauf koennte fehlerhafte Zeilen erzeugen. Landeten die sofort im Team-Laufwerk, saehen alle Kollegen sie sofort. Der Zwischenschritt gibt Ihnen die Gelegenheit zu pruefen, bevor etwas offiziell wird.

4. Wo Sie mit der Recherche anfangen

Es gibt zwei Betriebsarten. Die Wahl entscheidet, welche Firmen bearbeitet werden.

Betriebsart A: gaps - Luecken im Bestand schliessen

Das Tool nimmt Firmen, die **bereits in der Tabelle stehen**, bei denen aber Felder leer sind, und füllt diese Luecken. Es kommen keine neuen Firmen dazu.

Damit sollten Sie anfangen. Der Grund: Sie kennen diese Firmen und koennen sofort beurteilen, ob das Tool vernuenftige Ergebnisse liefert. Bei unbekannten Firmen wuessten Sie nicht, ob ein Fehler vorliegt oder die Information einfach neu ist.

Betriebsart B: new - neue Firmen entdecken

Das Tool durchsucht das Internet nach Firmen in den Marktsegmenten, die Sie vorgeben, und legt fuer jede neue Firma einen kompletten Datensatz an. Nutzen Sie das erst, wenn Sie mit den Ergebnissen aus Betriebsart A zufrieden sind.

	gaps (Standard)	new
Zweck	Bekannte Firmen vervollstaendigen	Unbekannte Firmen finden
Websuche	Nein	Ja
Neue Zeilen	Nur neue Produkte	Ganze Firmen mit allen Produkten
Empfehlung	Hier anfangen	Erst danach

Ihr allererster Lauf

Nehmen Sie **Betriebsart gaps mit fuenf Firmen**. Das dauert wenige Minuten, und Sie sehen am Ergebnis sofort, ob die Qualitaet stimmt. Der genaue Befehl steht in Kapitel 6.

Womit füllt das Tool die Luecken?

Welche Spalten als lueckenhaft gelten, steht in der Datei `config\targets.yaml`. Voreingestellt sind die vier Spalten mit dem niedrigsten Fuellgrad: `Product_name`, `All_Key_Categories`, `R-Strategies` und `Remarks`. Wie Sie das aendern, steht in Kapitel 8.

5. Voraussetzungen

Diese Dinge muessen **einmalig** eingerichtet sein. Danach nie wieder.

5.1 Was auf dem Rechner sein muss

Tippen Sie die Pruefbefehle nacheinander ins Terminal. Jeder muss eine Versionsnummer ausgeben.

Was	Wozu	Pruefbefehl
Python 3.11 oder neuer	Das Tool ist in Python geschrieben	<code>py --version</code>
Die claude-Anwendung	Liest und sortiert die Webseiteninhalte	<code>claude --version</code>
Git	Nur noetig fuer Kapitel 11	<code>git --version</code>
codex (<i>freiwillig</i>)	Zweitmeinung zur Kontrolle	<code>codex --version</code>

Kommt stattdessen *"Die Benennung ... wurde nicht als Name eines Cmdlet ... erkannt"*, fehlt das Programm und muss installiert werden.

5.2 Anmeldung - und warum es nichts extra kostet

Das Tool nutzt Ihr vorhandenes **Claude-Abonnement**. Es braucht **keinen kostenpflichtigen Zugangsschlüssel**, den man zusaetzlich kaufen muesste. Damit das funktioniert, muss die claude-Anwendung angemeldet sein.

Zum Pruefen oder Anmelden starten Sie sie einmal ohne Zusatz:

```
claude
```

Erscheint eine Eingabezeile, sind Sie angemeldet - mit `/exit` wieder heraus. Werden Sie nach einer Anmeldung gefragt, tippen Sie:

```
/login
```

und folgen den Anweisungen im Browser. Das ist einmalig noetig.

5.3 Programmbibliotheken installieren

Terminal oeffnen, in den Projektordner wechseln, dann einmalig:

```
py -m pip install -r requirements.txt
```

5.4 Der Selbsttest

Pruefen Sie zum Abschluss, ob alles bereit ist:

```
py -m cintel doctor
```

Erwartete Ausgabe:

CINTEL DOCTOR

```
=====
[ok]  claude-CLI gefunden: C:\Users\olafp\.local\bin\claude.EXE
[ok]  codex-CLI gefunden (Cross-Check moeglich)
[ok]  Taxonomie: config/taxonomy.yaml
[ok]  Ziele: config/targets.yaml
[--]  Master-DB: nicht angegeben (--master)
=====
Bereit.
```

Steht dort "Bereit.", koennen Sie loslegen.

Die Zeile mit [- -] ist kein Fehler - Sie haben hier nur noch keine Tabelle angegeben. Steht bei c laude-CLI dagegen [FEHL], gehen Sie zurueck zu Abschnitt 5.2.

6. Das Tool starten

6.1 Welche Tabelle nehmen Sie als Grundlage?

Datei	Beschreibung
...Master_DB_v2.2.xlsx auf H:\	Das Original im Team-Laufwerk. Enthält noch bekannte Fehler: kaputte Umlaute, Seitentitel statt Internetadressen, uneinheitliche Schreibweisen
data\outputs\...v2.2r.xlsx	Empfohlen. Dieselbe Tabelle, aber bereinigt: 1013 Korrekturen, danach keine Fehler mehr

6.2 Der erste Lauf - fuerf Firmen

Ein Befehl, eine Zeile. Alles kopieren, Eingabetaste:

```
py -m cintel run --master "data\outputs\Competitive_Intel_Master_DB_v2.2r.xlsx" --limit 5 --version 2.3
```

Was die Bestandteile bedeuten

Bestandteil	Bedeutung
py -m cintel	Starte das Programm cintel
run	Fuehre einen Recherchelauf aus
--master "..."	Welche Tabelle als Grundlage dient. Anführungszeichen sind noetig, weil der Pfad Leerzeichen enthaelt
--limit 5	Hoechstens fuerf Firmen. Schuetzt vor einem versehentlich riesigen Lauf
--version 2.3	Wie die Ergebnisdatei heissen soll: ... v2.3 .xlsx

Die Bestandteile mit zwei Bindestrichen heissen *Schalter*. Ihre Reihenfolge ist beliebig.

6.3 Was Sie auf dem Bildschirm sehen

```
Master-DB: 918 Zeilen, 375 Firmen
Modus 'gaps': 5 Firmen zur Anreicherung ausgewaehlt.
[1/5] 3D Spark - https://www.3dspark.de
      6 Seiten -> 1 Firmenzeile + 7 Produkte
[2/5] aPriori - https://www.apriori.com/blog/...
      12 Seiten -> 1 Firmenzeile + 7 Produkte
[3/5] Autodesk - https://www.autodesk.com/bim-360
      abgelehnt: Homepage nicht erreichbar (durch robots.txt untersagt)
[4/5] CAESSES (Friendship Systems) - https://www.caeses.com
      1 Seiten -> 1 Firmenzeile + 1 Produkte
[5/5] Celus - https://celus.io
      10 Seiten -> 1 Firmenzeile + 2 Produkte

11 neue Zeilen, 1 bestehende Zeilen ergaenzt, 12 Felder gefuehlt, 73 uebersprungen

Neue Version : data\outputs\Competitive_Intel_Master_DB_v2.3.xlsx
Lauf-Ordner  : data\outputs\run_20260818_230425
```

- **abgelehnt** ist kein Programmfehler. Manche Firmen - etwa Autodesk, Cadence oder Aspen - verbieten in ihrer Datei *robots.txt* ausdruecklich das automatische Auslesen. Das respektieren wir. In einer Stichprobe von 60 Firmen betraf das 5.
- **uebersprungen** heisst: das Feld war schon gefuehlt oder das Produkt stand bereits in der Tabelle. Auch das ist gewollt.
- Rechnen Sie mit etwa **30 bis 60 Sekunden pro Firma**.

6.4 Erst schauen, nichts schreiben

Wenn Sie den Ablauf sehen wollen, ohne dass eine Excel-Datei entsteht, haengen Sie `--dry-run` an. Die Berichtsdateien werden trotzdem erzeugt:

```
py -m cintel run --master "data\outputs\Competitive_Intel_Master_DB_v2.2r.xlsx" --limit 5 --dry-run
```

Gleiche Versionsnummer ueberschreibt die Datei

Starten Sie zweimal mit `--version 2.3`, wird die vorhandene Datei ersetzt. Vergeben Sie fuer jeden Lauf, den Sie behalten wollen, eine neue Nummer: 2.3, dann 2.4, dann 2.5.

7. Die Ergebnisse verstehen

Alles landet in diesem Ordner:

```
C:\Users\olafp\Desktop\Arbeitsordner\260818_Business_Intell_Comp_research_Tool\data\outputs\
```

Diesen Ordner oeffnen Sie am schnellsten aus dem Terminal heraus - der Windows-Explorer geht auf:

```
explorer data\outputs
```

7.1 Die Excel-Datei

Competitive_Intel_Master_DB_v2.3.xlsx ist die vollstaendige Tabelle - alter Bestand **plus** die neuen Zeilen. Die neuen Zeilen stehen **ganz unten**. Mit **Strg + Ende** springen Sie in Excel sofort dorthin.

7.2 Der Lauf-Ordner

Zu jedem Lauf gehoert ein Ordner run_JJJJMMTT_HHMMSS mit fuenf Dateien. **Diese Dateien sind Ihre Qualitaetskontrolle** - schauen Sie hinein, bevor Sie das Ergebnis weiterverwenden:

Datei	Was drinsteht	Wann Sie sie brauchen
report.md	Zusammenfassung: was gefuehlt, was abgelehnt, was uebersprungen wurde	Immer zuerst lesen. Mit Notepad oder Word zu oeffnen
rejected.csv	Jede abgelehnte Firma mit Begrueundung	Wenn eine Firma fehlt, die Sie erwartet haben
new_rows.csv	Die neuen Zeilen einzeln	Zum schnellen Durchsehen ohne Excel
sources.csv	Jede aufgerufene Seite mit Status	Wenn Sie belegen wollen, woher eine Angabe stammt
plan.json	Technisches Protokoll	Bei Rueckfragen an die Technik

7.3 Woran Sie gute Ergebnisse erkennen

Oeffnen Sie die neue Excel-Datei und pruefen Sie an den untersten Zeilen diese fuenf Punkte:

- 1 Sind die Produktnamen echt?** Es muessen benannte Produkte sein ("Fusion 360"), keine Werbefloskeln ("Unsere Loesung").
- 2 Stimmt die Company_ID?** Alle Zeilen einer Firma muessen dieselbe Nummer tragen wie die schon vorhandenen Zeilen dieser Firma.
- 3 Ist die Spalte URL eine echte Internetadresse?** Sie muss mit `https://` beginnen.
- 4 Sind die Kategorien aus der Liste?** In *Sub Category B* duerfen nur die 30 vorgesehenen Werte stehen, keine frei erfundenen.
- 5 Wurde nichts ueberschrieben?** Ihre Einschaeztungen in *Competitor_Tier* und *Beachhead_Relevanz* muessen unveraendert sein.

Punkt 5 garantiert das Tool von sich aus

Bei bereits bekannten Firmen werden **ausschliesslich leere Felder** gefuellt. Ihre von Hand gepflegten Bewertungen bleiben in jedem Fall stehen - auch wenn die Auswertung etwas anderes vorschlaegt. Was aus diesem Grund nicht uebernommen wurde, listet *report.md* auf.

7.4 Die Tabelle jederzeit pruefen lassen

Dieser Befehl durchsucht eine beliebige Fassung nach Fehlern - kaputte Umlaute, falsche Internetadressen, doppelte Zeilen, unbekannte Kategorien:

```
py -m cintel validate --master "data\outputs\Competitive_Intel_Master_DB_v2.3.xlsx"
```

Und dieser bereinigt gefundene Fehler in eine neue Datei. Mit `--dry-run` zeigt er zunaechst nur an, was er aendern wuerde:

```
py -m cintel repair --master "<pfad zur xlsx>" --dry-run
```

8. Qualitaet und Verhalten anpassen

Alle Stellschrauben liegen in Textdateien im Ordner config. Sie oeffnen sie mit einem normalen Texteditor - Rechtsklick auf die Datei, *Oeffnen mit, Editor*.

Zwei Regeln beim Bearbeiten dieser Dateien

1. Nur Leerzeichen zum Einruecken verwenden, niemals die Tabulatortaste. 2. Die Einrueckung genau so beibehalten, wie sie ist. Beides fuehrt sonst zu einer Fehlermeldung beim Start.

8.1 "Es bearbeitet die falschen Firmen"

Datei: config\targets.yaml. Der Abschnitt gaps bestimmt die Auswahl:

```
gaps:
  target_columns:
    - "Product_name"
    - "All_Key_Categories of this company"
    - "R-Strategies"
    - "Remarks Product/solution"
  only_incomplete: true
  tier_priority:
    - "Tier 1 - Direkt"
    - "Tier 2 - Nachbar"
    - "Tier 3 - Beobachten"
  require_url: true
```

- **Andere Spalten fuellen:** Zeilen unter target_columns aendern. Die Spaltennamen muessen **exakt** so geschrieben sein wie in der Excel-Kopfzeile.
- **Nur bestimmte Wettbewerber:** Bei tier_priority die unerwuenschten Zeilen loeschen. Wer nur Tier 1 bearbeiten will, laesst nur diese eine Zeile stehen.

8.2 "Die Ergebnisse sind zu ungenau"

Beobachtung	Was Sie tun
Zu wenige Produkte gefunden	In targets.yaml max_pages_per_company von 12 auf 20 erhoehen. Das Tool liest dann mehr Unterseiten
Ungenauere oder erfundene Angaben	Auf das staerkere Modell wechseln: unter llm: den Eintrag model: "sonnet" auf "opus" setzen. Langsamer, aber gruendlicher
Zu viele unsichere Angaben landen in der Tabelle	Die Confidence-Schwelle anheben. Sie steht in cintel\merge.py als min_confidence: float = 0.35. Wert 0.6 laesst nur gut belegte Angaben durch
Zahlen sollen doppelt geprueft werden	Beim Start --cross-check codex anhaengen. Gruendungsjahr, Mitarbeiterzahl und Standort werden dann unabhaengig nachgeprueft und Abweichungen gemeldet. Setzt voraus, dass codex login einmal ausgefuehrt wurde
Gruendungsjahr und Mitarbeiterzahl bleiben leer	Diese Angaben stehen auf der Ueber-uns-Seite. Das Tool ruft sie bevorzugt ab, aber nicht jede Firma veroeffentlicht sie. Finanzierungsrunde und Umsatz stehen fast nie auf der eigenen Website - die muessen von Hand ergaenzt werden

Beobachtung	Was Sie tun
Marktsegmente fuer Betriebsart <i>new</i> aendern	Im Abschnitt <i>new</i> : die Eintraege unter <i>key_categories</i> , <i>sub_categories</i> und <i>regions</i> anpassen - oder gleich ein eigenes Profil anlegen, siehe Kapitel 10

8.3 "Es erfindet neue Kategorien"

Datei: `config\taxonomy.yaml`. Sie enthaelt die **erlaubten Werte**. Das Tool darf nichts anderes schreiben - genau deshalb gibt es diese Datei. Frueher war die Tabelle auf 34 Hauptkategorien angewachsen statt der vorgesehenen 8.

Eine neue Sub-Kategorie fuegen Sie so hinzu - beachten Sie den Bindestrich und die Anfuhrungszeichen:

```
sub_categories:
  - "3D Content & Visualization"
  - "AI & Surrogate Simulation"
  - "Meine neue Kategorie"           # <- neue Zeile
```

Damit die neue Kategorie auch verwendet wird, tragen Sie sie zusaetzlich unter *legend*: bei der passenden Hauptkategorie ein.

Schreibvarianten umlenken statt neu anlegen

Taucht in der Tabelle eine abweichende Schreibweise auf, die eigentlich einen vorhandenen Wert meint, tragen Sie sie unter *sub_category_aliases* ein. Beispiel: "AI & ML Tools": "Engineering Copilots & AI Assistants". Der Befehl *repair* raeumt sie dann automatisch auf.

9. Spalten ergaenzen

Angenommen, die Tabelle soll eine neue Spalte **Funding_Total** bekommen. Dafuer sind vier Schritte noetig - in genau dieser Reihenfolge.

Warum das nicht mit Excel allein geht

Das Tool kennt die 31 Spalten der Tabelle als festen Vertrag. Legen Sie eine Spalte nur in Excel an, wird sie nie gefuellt - und das Tool bricht beim naechsten Lauf mit einer Schema-Meldung ab.

Schritt 1: Spalte in der Excel-Datei anlegen

Neue Spalte **rechts an das Ende** setzen, also hinter *Beachhead_Relevanz*. In die Kopfzeile den Namen schreiben: `Funding_Total`.

Schritt 2: Spalte im Vertrag eintragen

Datei `cintel\schema.py` oeffnen. Ganz oben steht die Liste `COLUMNS`. Am Ende - vor der schliessenden eckigen Klammer - eine Zeile ergaenzen:

```
( "beachhead",      "Beachhead_Relevanz"),  
( "funding_total", "Funding_Total"),      # <- neue Zeile  
)
```

Links der interne Kurzname in Kleinbuchstaben, rechts die Excel-Kopfzeile in **exakt** derselben Schreibweise. Beides in Anfuhrungszeichen, am Ende ein Komma.

Schritt 3: Feld zur Auswertung hinzufuegen

Datei `cintel\extract.py`. In der Funktion `build_schema` steht der Block `company_schema`. Dort bei `"properties"` ergaenzen:

```
"funding_total": {"type": "string"},
```

Danach, weiter unten in der Funktion `_to_records`, im Block `company_values`:

```
"funding_total": _clean(info.get("funding_total")),
```

Schritt 4: Pruefen

Die mitgelieferten Tests stellen sicher, dass Sie nichts zerbrochen haben. Sie muessen alle durchlaufen:

```
py -m pytest tests/ -q
```

Erwartete Ausgabe am Ende: `passed` - es darf kein `failed` auftauchen. Dann ein kleiner Testlauf mit zwei Firmen:

```
py -m cintel run --master "data\outputs\Competitive_Intel_Master_DB_v2.2r.xlsx" --limit 2 --dry-run
```

Erscheint eine Meldung ueber fehlende Pflichtspalten?

Dann stimmt der Spaltenname in `schema.py` nicht genau mit der Excel-Kopfzeile ueberein. Achten Sie auf Gross- und Kleinschreibung, Unterstriche und versehentliche Leerzeichen am Ende.

10. Laeufe automatisieren

Bisher haben Sie jeden Lauf von Hand gestartet und alle Einstellungen als Schalter mitgegeben. Fuer wiederkehrende Recherchen gibt es einen bequemerem Weg: **Profile**.

10.1 Was ein Profil ist

Ein Profil ist ein gespeicherter Suchauftrag. Es haelt fest, welche Branche, welche Funktionsbereiche, welche Region und welchen Reifegrad Sie suchen. Statt eines langen Befehls mit vielen Schaltern rufen Sie das Profil dann nur beim Namen.

Welche Profile es gibt, zeigt dieser Befehl:

```
py -m cintel profiles
```

Profil	Was es tut
bestand-luecken	Fuellte Luecken bei bekannten Firmen. Der Standardlauf fuer die laufende Pflege
tier1-tiefenpruefung	Nur direkte Wettbewerber, mehr Unterseiten, staerkeres Modell. Gruendlich und langsamer
lca-startups-dach	Sucht neue Anbieter fuer Oekobilanz und CO2 im deutschsprachigen Raum
engineering-ki-europa	Sucht neue KI-Assistenten fuer Konstruktion und Simulation in Europa
angebotsphase-rfq	Sucht Anbieter rund um Angebotskalkulation und Anfragebearbeitung

10.2 Ein Profil starten

```
py -m cintel run --master "data\outputs\Competitive_Intel_Master_DB_v2.2r.xlsx" --profile lca-startups-dach
```

Alles Weitere - Branche, Funktionsbereiche, Region, Reifegrad, Firmenzahl - steckt im Profil. Einzelne Angaben lassen sich trotzdem ueberschreiben; ein mitgegebener Schalter hat immer Vorrang:

```
py -m cintel run --master "<datei.xlsx>" --profile lca-startups-dach --limit 3 --dry-run
```

10.3 Ein eigenes Profil anlegen

Datei config\profiles.yaml oeffnen und einen neuen Abschnitt anhaengen. Dieses Beispiel sucht Medizintechnik-Anbieter in Europa:

```

medtech-europa:
  description: >
    Sucht Anbieter mit Schwerpunkt Medizintechnik in Europa.
  mode: new
  limit: 12
  key_categories:
    - "4. Lifecycle & Data Management (PLM/PDM)"
  sub_categories:
    - "PLM & PDM Platforms"
    - "MBSE & Systems Engineering"
  regions:
    - "Europe"
  stages:
    - "Series A"
    - "Series B"
  inclusion_criteria: >
    Software mit nachweisbarem Bezug zur Medizintechnik.
  exclusion_criteria: >
    Reine Beratung ohne eigenes Produkt.

```

- Die Einrückung muss genau stimmen: der Profilname zwei Leerzeichen eingerückt, seine Eigenschaften vier.
- Alle Kategorienamen müssen **exakt** so geschrieben sein wie in config\taxonomy.yaml.
- Die Sub-Kategorien müssen zu den gewählten Hauptkategorien passen. Welche wohin gehören, steht in taxonomy.yaml im Abschnitt legend.
- Prüfen Sie danach mit `py -m cintel profiles`. Stimmt etwas nicht, steht dort **FEHLER** mit Begründung - noch bevor ein Lauf startet.

Warum die Zuordnung wichtig ist

Passt eine Sub-Kategorie nicht zur Hauptkategorie, würde die Suche sie stillschweigend verwerfen und stattdessen über *alle* Sub-Kategorien laufen. Sie bekommen Ergebnisse - nur eben zu etwas anderem als gedacht. Deshalb meldet die Profilprüfung diesen Fall als Fehler.

10.4 Zeitgesteuert laufen lassen

Weil ein Profillauf immer derselbe kurze Befehl ist, kann Windows ihn selbsttätig starten - etwa jeden Montagmorgen. Dafür liegen zwei Skripte bereit.

Einen Zeitplan einrichten

```
.\scripts\Register-CintelSchedule.ps1 -ProfileName bestand-luecken -Schedule Weekly -DayOfWeek Monday -Time 06:30
```

Das legt in der Windows-Aufgabenplanung einen Eintrag an. Ab jetzt läuft die Recherche jeden Montag um 6:30 Uhr, und Sie finden das Ergebnis im gewohnten Ausgabeordner.

Schalter	Bedeutung
-ProfileName	Welches Profil laufen soll
-Schedule	Daily, Weekly oder Monthly
-DayOfWeek	Nur bei Weekly, z.B. Monday

Schalter	Bedeutung
-Time	Uhrzeit im Format HH:mm
-Remove	Entfernt den Zeitplan wieder

Sofort testen, ohne auf den Termin zu warten

```
Start-ScheduledTask -TaskName "cintel - bestand-luecken"
```

Zeitplan wieder entfernen

```
.\scripts\Register-CintelSchedule.ps1 -ProfileName bestand-luecken -Remove
```

10.5 Was Sie dabei wissen muessen

Der Rechner muss laufen und Sie muessen angemeldet sein

Die Aufgabe startet unter Ihrem Windows-Konto, weil die Anmeldung der claude-Anwendung daran haengt. Ist der Rechner aus oder sind Sie abgemeldet, faellt der Termin aus. Windows holt ihn beim naechsten Anmelden nach.

Jeder automatische Lauf schreibt ein Protokoll. Dort steht, was passiert ist - auch wenn Sie nicht dabei waren:

```
explorer data\logs
```

Das Runner-Skript prueft vor dem Start zwei Dinge und bricht mit verstaendlicher Meldung ab, wenn etwas fehlt: ob die Master-Tabelle erreichbar ist - bei Laufwerk H: kann Google Drive getrennt sein - und ob die claude-Anwendung vorhanden ist.

Falls ein Skript nicht starten will

Meldet PowerShell, die Ausfuehrung von Skripten sei deaktiviert, wurde die Datei als "aus dem Internet" eingestuft. Einmalig freigeben mit: Unblock-File .\scripts*.ps1

Ein sinnvoller Rhythmus

Profil	Vorschlag	Warum
bestand-luecken	woechentlich, Montag frueh	Haelt die vorhandenen Firmen aktuell
lca-startups-dach	monatlich	Neue Startups tauchen nicht woechentlich auf
engineering-ki-europa	monatlich	dasselbe
tier1-tiefenpruefung	von Hand, vor Quartalsberichten	Dauert laenger und will begutachtet werden

Automatisch heisst nicht ungeprueft

Auch ein zeitgesteuerter Lauf schreibt nur in den Ausgabeordner auf Ihrem Rechner. Ob das Ergebnis nach H: ins Team-Laufwerk wandert, entscheiden weiterhin Sie - nach einem Blick in *report.md*.

11. Aenderungen ueber GitHub einbringen

GitHub ist der gemeinsame Ablageort fuer den Programmcode. Er sorgt dafuer, dass jede Aenderung nachvollziehbar ist und rueckgaengig gemacht werden kann. Erinnerung aus Kapitel 3: **Der Abgleich passiert nicht von selbst** - Sie stossen ihn an.

11.1 Die vier Begriffe, die Sie brauchen

Begriff	Erklaerung in einem Satz
Branch (Zweig)	Eine Arbeitskopie, in der Sie etwas aendern, ohne die funktionierende Fassung anzufassen
Commit	Ein gespeicherter Zwischenstand mit Notiz, was Sie geaendert haben
Push	Ihre Commits zu GitHub hochladen
Pull Request (PR)	Der Antrag: "Bitte uebernehmt meine Aenderung in die Hauptfassung". Hier wird geprueft und besprochen

11.2 Der vollstaendige Ablauf

Immer dieselben sechs Schritte. Alle Befehle im Projektordner eingeben.

Schritt 1 — Auf den aktuellen Stand gehen

```
git checkout main
git pull
```

Schritt 2 — Eigenen Zweig anlegen

```
git checkout -b feat/funding-spalte
```

Der Name ist frei wahlbar. Ueblich sind die Vorsilben `feat/` fuer Neues und `fix/` fuer Fehlerbehebungen.

Schritt 3 — Aendern und pruefen

Jetzt die Dateien bearbeiten. Danach **unbedingt** die Tests laufen lassen - vor dem Speichern, nicht danach:

```
py -m pytest tests/ -q
```

Schritt 4 — Zwischenstand speichern

```
git add -A
git commit -m "feat: Spalte Funding_Total ergaenzt"
```

Die Notiz hinter `-m` beschreibt kurz, was Sie geaendert haben. Uebliche Vorsilben: `feat:` neue Funktion, `fix:` Fehlerbehebung, `docs:` Dokumentation, `test:` Tests.

Schritt 5 — Hochladen

```
git push -u origin feat/funding-spalte
```

Schritt 6 — Pull Request eroeffnen

```
gh pr create --fill
```

Der Befehl gibt eine Internetadresse aus. Öffnen Sie sie - dort sehen Sie Ihre Änderung und die automatische Prüfung.

11.3 Die automatische Prüfung

Sobald der Pull Request offen ist, prüft GitHub den Code selbsttätig auf drei Python-Versionen. Status abfragen:

gh pr checks	
Anzeige	Bedeutung
pass	Alles in Ordnung - der PR kann übernommen werden
fail	Etwas ist kaputt. Auf die angezeigte Adresse klicken, Meldung lesen, korrigieren, dann Schritt 3 bis 5 wiederholen
pending	Läuft noch. Einen Moment warten und erneut abfragen

11.4 Übernehmen und aufräumen

Wenn alle Prüfungen bestanden sind und der Inhalt passt:

```
gh pr merge --squash --delete-branch
```

Danach zurück auf die Hauptfassung:

```
git checkout main  
git pull
```

Diese Dateien dürfen nie nach GitHub

Excel-Dateien, der Ordner data und die Datei .env sind ausgeschlossen - dort stehen Wettbewerbsdaten und Zugangsdaten. Die Sperre ist eingerichtet. Prüfen können Sie es vor dem Hochladen mit `git status`: taucht dort eine .xlsx auf, halten Sie an und fragen nach.

12. Fehlermeldungen nachschlagen

Meldung auf dem Bildschirm	Was zu tun ist
Die Benennung "py" wurde nicht ... erkannt	Python fehlt, oder das Terminal steht im falschen Ordner. Erst py --version pruefen, dann Kapitel 2.4
Not logged in · Please run /login	Die claude-Anwendung ist nicht angemeldet. claude starten, /login eingeben
claude-CLI wurde nicht gefunden	Die Anwendung ist nicht installiert oder dem System nicht bekannt. Mit claude --version pruefen
Master-DB nicht gefunden	Der Pfad hinter --master stimmt nicht. Auf die Anfuhrungszeichen achten und pruefen, ob Laufwerk H: verbunden ist
Pflichtspalte(n) nicht gefunden	Die Excel-Datei hat nicht die erwarteten 31 Spalten. Wurde eine Spalte umbenannt oder geloesch? Siehe Kapitel 9
Profil '...' gibt es nicht	Tippfehler im Profilnamen. Liste anzeigen mit py -m cintel profiles
codex ist nicht angemeldet	Betrifft nur --cross-check codex. Einmalig codex login ausfuehren. Der Lauf selbst geht trotzdem vollstaendig durch - die Zweitpruefung entfaellt lediglich
Sub Category '...' passt zu keiner der gewaehlten Key Categories	In Ihrem Profil gehoert eine Sub-Kategorie nicht zur gewaehlten Hauptkategorie. Die Meldung nennt, wohin sie gehoert. Siehe Kapitel 10.3
durch robots.txt untersagt	Kein Fehler. Diese Firma verbietet automatisches Auslesen. Die Angaben muessen von Hand ergaenzt werden
Firmenname nicht auf der Seite gefunden	Die hinterlegte Adresse fuehrt zur falschen Seite. Die Spalte URL in der Tabelle pruefen und berichtigen
Homepage nicht erreichbar	Die Adresse existiert nicht mehr oder ist falsch geschrieben. Im Browser aufrufen und in der Tabelle korrigieren
0 neue Zeilen	Kein Fehler - es gab nichts zu ergaenzen. report.md im Lauf-Ordner nennt den Grund fuer jedes uebersprungene Feld
Die Ausfuehrung von Skripts ist deaktiviert	Betrifft die .ps1-Skripte aus Kapitel 10. Einmalig freigeben mit Unblock-File .\scripts*.ps1
Umlaute erscheinen als Kaestchen	Vor dem Befehl einmalig chcp 65001 eingeben. Betrifft nur die Anzeige, nicht die Daten

13. Befehlsuebersicht

Alle Befehle werden im Projektordner eingegeben. Dorthin gelangen Sie mit:

```
cd "C:\Users\olafp\Desktop\Arbeitsordner\260818_Business Intell_Comp research_Tool"
```

Recherche

Zweck	Befehl
Voraussetzungen pruefen	py -m cintel doctor
Profile anzeigen	py -m cintel profiles
Erster Lauf, 5 Firmen	py -m cintel run --master "data\outputs\...v2.2r.xlsx" --limit 5 --version 2.3
Lauf mit Profil	py -m cintel run --master "<datei.xlsx>" --profile bestand-luecken
Nur anschauen, nichts schreiben	... --dry-run
Neue Firmen suchen	... --mode new --limit 10
Mit Zweitpruefung	... --cross-check codex
Nur Webseiten laden	... --crawl-only
Ohne Internet, nur Zwischenspeicher	... --offline

Tabelle pruefen und bereinigen

Zweck	Befehl
Auf Fehler pruefen	py -m cintel validate --master "<datei.xlsx>"
Bereinigung anzeigen	py -m cintel repair --master "<datei.xlsx>" --dry-run
Bereinigung ausfuehren	py -m cintel repair --master "<datei.xlsx>" --version 2.2r
Struktur und Fuellgrade ansehen	py scripts\inspect_master_db.py "<datei.xlsx>"

Automatisierung

Zweck	Befehl
Profillauf von Hand starten	.\scripts\Run-CintelProfile.ps1 -ProfileName bestand-luecken
Zeitplan einrichten	.\scripts\Register-CintelSchedule.ps1 -ProfileName bestand-luecken -Schedule Weekly -DayOfWeek Monday -Time 06:30
Zeitplan sofort testen	Start-ScheduledTask -TaskName "cintel - bestand-luecken"
Zeitplan entfernen	.\scripts\Register-CintelSchedule.ps1 -ProfileName bestand-luecken -Remove
Protokolle ansehen	explorer data\logs

Entwicklung und GitHub

Zweck	Befehl
Tests ausfuehren	<code>py -m pytest tests/ -q</code>
Code pruefen	<code>py -m ruff check cintel tests scripts</code>
Neuen Zweig anlegen	<code>git checkout -b feat/mein-thema</code>
Was habe ich geaendert?	<code>git status</code>
Zwischenstand speichern	<code>git add -A</code> danach <code>git commit -m "feat: ..."</code>
Hochladen	<code>git push -u origin feat/mein-thema</code>
Pull Request eroeffnen	<code>gh pr create --fill</code>
Pruefstatus ansehen	<code>gh pr checks</code>
Uebernehmen	<code>gh pr merge --squash --delete-branch</code>
Handbuch neu erzeugen	<code>py docs\build_handbuch.py</code>

Im Zweifel gilt

Sie koennen mit diesem Tool nichts zerstoeren. Die Master-Datei auf H:\ wird ausschliesslich gelesen. Jedes Ergebnis ist eine neue Datei mit eigener Versionsnummer. Wenn ein Lauf misslingt, loeschen Sie die entstandene Datei und starten neu.